Digital VLSI Design

Assignment 1

4-bit Unsigned Array Multiplier from RTL to GDSII

SkyWater SKY130 (`sky130_fd_sc_hd`) | Icarus Verilog, Yosys, OpenSTA, LibreLane

Team member	**Roll number**
Ande Karthik	2023102009
Kalluri Roshan Lal	2023102010

Contents

1 Introduction **3**

2 Part A: RTL Design and Verification **3**
2.1 Full adder . . . 3
2.2 How the array multiplier works . . . 3
2.3 Block diagram . . . 5
2.4 Testbench . . . 5
2.5 Simulation results . . . 6
2.6 Hand-worked example (task 5) . . . 6
2.7 Waveform . . . 7

3 Part B: Logic Synthesis with Yosys **8**
3.1 Synthesis script . . . 8
3.2 Part B results . . . 8
3.3 Full cell breakdown . . . 9
3.4 Gate-level (netlist) simulation . . . 9

4 Part C: Static Timing Analysis with OpenSTA **10**
4.1 STA script and constraints . . . 10
4.2 Finding the maximum frequency . . . 10
4.3 Part C results . . . 11
4.4 Critical path . . . 11
4.5 Comparison with the $2N = 8$ full-adder estimate . . . 11

5 Part D: Physical Design with LibreLane **12**
5.1 The flow . . . 12
5.2 Timing closure: every attempt . . . 13
5.3 Part D results . . . 14
5.4 Timing at every signoff corner . . . 14
5.5 Post-route F_{max} compared with Part C . . . 15
5.6 Cells added by place and route . . . 15
5.7 Final layout . . . 17

6 Answers to the Report Questions **20**
Q1. Logic cell count . . . 20
Q2. Critical path . . . 20
Q3. Flip-flop area . . . 20
Q4. Post-route Fmax and added cells . . . 21
Q5. DRC and LVS . . . 21
Q6. Scaling to 8 bits . . . 22

7 Individual Contributions **22**

1 Introduction

GitHub link: `https://github.com/KarthiK-Ande-22/DVD-Assignment-1`

We designed a registered 4-bit × 4-bit unsigned array multiplier (8-bit product, 2-cycle latency, one result per cycle) and took it from Verilog RTL to a GDSII layout on SkyWater SKY130 (`sky130_fd_sc_hd`) with open-source tools. Each part checks one aspect of the design:

Part	Result
A. RTL design and verification	256/256 input pairs correct; an injected adder bug gives 95 errors
B. Logic synthesis	73 cells, 805.77 μm^2; netlist passes the same 256 tests
C. Static timing analysis	T_{min} = 2.596 ns, F_{max} = 385.21 MHz (typical corner)
D. Physical design	0 DRC, 0 LVS at a 7.0 ns clock; post-route F_{max} = 150.70 MHz

Table 1: Parts of the flow and their results.

The Part A testbench also verifies the Part B netlist, Part C analyses that netlist, and $1.2 \times T_{min}$ from Part C sets the first clock period of Part D. Source files, reports, the GDSII and every command (`README.md`) are in the repository linked above. Table 2 lists the tools used.

Stage	Tool and version
Simulation, waveforms	Icarus Verilog 14.0 (`-g2012`), GTKWave 4.0.0
Logic synthesis	Yosys 0.70 with ABC
Static timing analysis	OpenSTA 3.1.0
RTL-to-GDSII flow	LibreLane 3.1.0.dev3, Classic flow (OpenROAD, Magic, KLayout, Netgen)
PDK and library	`sky130A`, `sky130_fd_sc_hd`; typical corner `tt_025C_1v80` (typical transistors, 25 °C, 1.80 V) for Parts B and C
Environment	Ubuntu 24.04 host; IIC-OSIC-TOOLS Docker container for all parts (`sky130A` selected explicitly, as the container defaults to IHP)

Table 2: Tools used in each stage of the flow.

2 Part A: RTL Design and Verification

2.1 Full adder

`rtl/full_adder.v` defines module `full_adder` with inputs `a`, `b`, `cin` and outputs `sum`, `cout`, written only with `assign` statements:

$$\texttt{sum} = a \oplus b \oplus cin \qquad \text{(1 when an odd number of inputs are 1)}$$
$$\texttt{cout} = a\,b + b\,cin + a\,cin \qquad \text{(majority: 1 when two or more inputs are 1)}$$

2.2 How the array multiplier works

Binary multiplication follows the same long-multiplication method used on paper, except that every digit is 0 or 1. Bit b_i of the multiplier B decides what appears in row i: the partial product pp_i is

a copy of A when $b_i = 1$ and all zeros when $b_i = 0$, and it is worth 2^i, i.e. shifted i places to the left. Selecting one bit is a single AND, $pp_i[j] = a[j] \cdot b[i]$, so the four 4-bit partial products need $4 \times 4 = 16$ AND gates. The product is the sum of the four shifted partial products.

Instead of one large adder, the array accumulates the partial products one row at a time (Table 3):

- **Row 0** contains only AND gates. Its least significant bit $pp_0[0]$ has nothing to be added to, so it is already the product bit $p[0]$; its upper three bits move down to row 1.
- **Rows 1, 2 and 3** are each a 4-bit *ripple-carry adder*: a chain of 4 full adders in which every carry-out feeds the carry-in of the next adder, and the first carry-in is 0. Row i adds pp_i to the 4 bits arriving from the row above. Three rows of four give the 12 full adders.
- Once a row has added, its lowest sum bit can no longer change, so it is a finished product bit ($p[1]$, $p[2]$, $p[3]$); the other three sum bits and the final carry move down to the next row. Row 3 has no row below it, so its remaining outputs form the upper half of the product, $p[7{:}4]$.

Row	**Inputs to full adder j**	**Outputs**
0	no adders (AND gates only)	$p[0] = pp_0[0]$; $r_0 = \{0, pp_0[3], pp_0[2], pp_0[1]\}$
1	$pp_1[j], r_0[j], c_1[j]$ $(c_1[0] = 0)$	$s_1[j], c_1[j{+}1]$; $p[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3], s_1[2], s_1[1]\}$
2	$pp_2[j], r_1[j], c_2[j]$ $(c_2[0] = 0)$	$s_2[j], c_2[j{+}1]$; $p[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3], s_2[2], s_2[1]\}$
3	$pp_3[j], r_2[j], c_3[j]$ $(c_3[0] = 0)$	$s_3[j], c_3[j{+}1]$; $p[3] = s_3[0]$; $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$

Table 3: Connections of the array ($j = 0 \ldots 3$; the first carry into each row is $c_i[0] = 0$).

RTL implementation. `rtl/mult_array.v` has exactly the required ports:

- `clk`: the clock; all registers update on its rising edge.
- `rst_n`: asynchronous, active-low reset. When it goes to 0, every register is cleared to 0 immediately, without waiting for a clock edge.
- `a[3:0]`, `b[3:0]`: the two operands.
- `p[7:0]`: the product, driven directly by a register.

Inside the module the data flows through four stages:

1. On each rising edge the operands are stored in the 4-bit input registers `a_reg` and `b_reg` (8 flip-flops).
2. The partial products are formed from the registered values, `ppi = a_reg & {4{b_reg[i]}}`, which gives the 16 AND gates.
3. Twelve `full_adder` instances, `fa10` to `fa33` (row, position), are connected exactly as in Table 3. The carry chains `c1`, `c2`, `c3` have their bit 0 tied to 0.
4. The combinational result {`c3[4]`, `s3[3:0]`, `s2[0]`, `s1[0]`, `pp0[0]`} is stored in the output register `p` (8 flip-flops) on the next rising edge.

This gives 16 flip-flops in total, and the `*` operator is not used anywhere in the RTL.

Latency and throughput. Because both the inputs and the output are registered, an operand pair applied before a rising edge is captured at that edge (edge 1), passes through the AND gates and adder rows during the following clock period, and its product is captured into `p` at the next edge (edge 2). The *latency* is therefore 2 clock cycles. The *throughput* is one product per cycle: while one product is being computed between the two register stages, the input registers can already accept the next operand pair. The structure is shown in Figure 1.

2.3 Block diagram

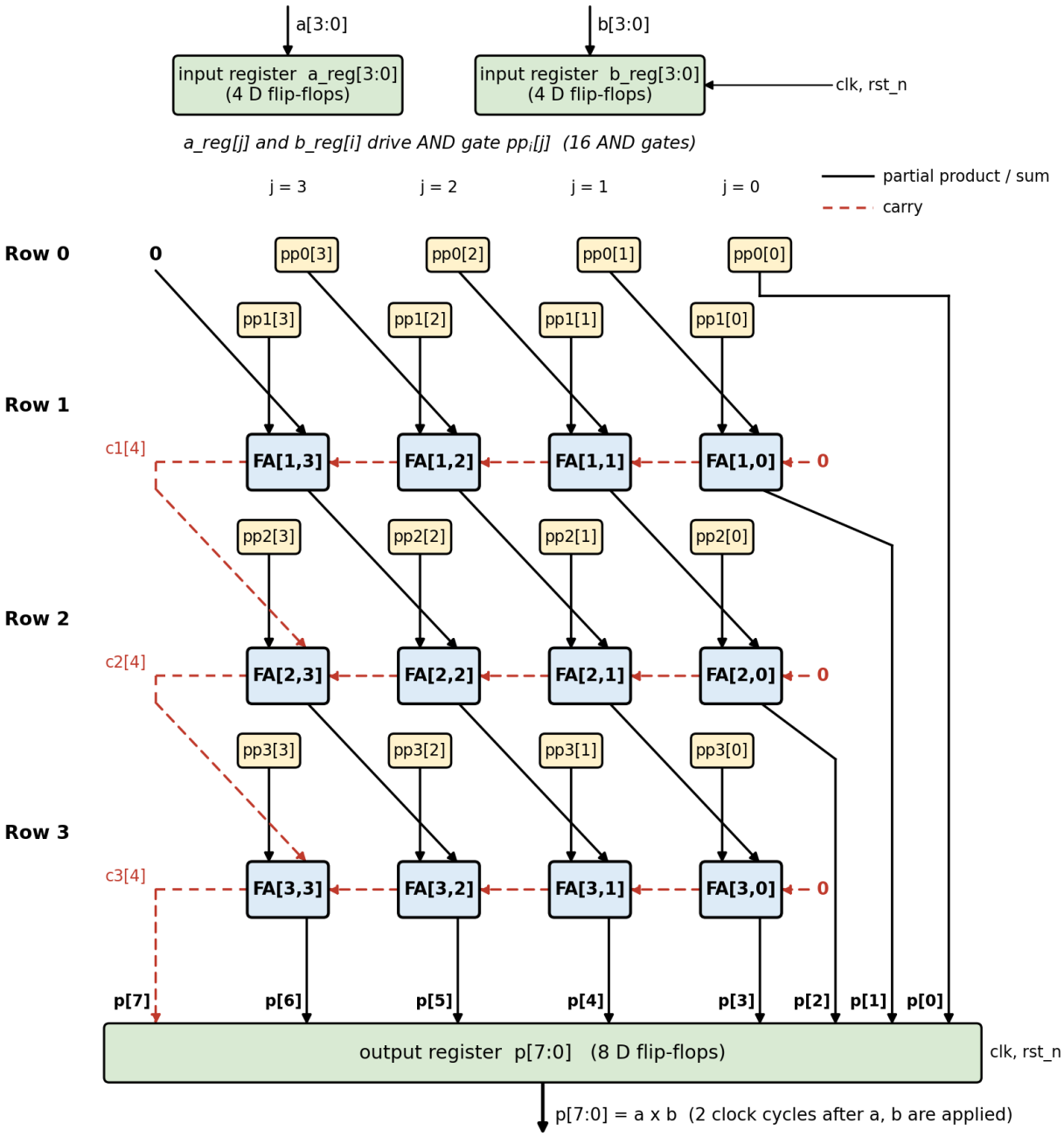


Figure 1: Block diagram of `mult_array`: input registers `a_reg`/`b_reg` (8 flip-flops), 16 AND gates $pp_i[j]$, 12 full adders FA$[i,j]$ in three ripple-carry rows, and the output register `p` (8 flip-flops). Each row passes its upper sum bits diagonally into the next row and its final carry $c_i[4]$ into FA$[i{+}1,3]$; carries (red, dashed) ripple from right to left.

2.4 Testbench

`tb/tb_mult.v` (module `tb_mult`) is an exhaustive, self-checking testbench: it applies every input pair, computes the expected product itself, and reports the result without manual inspection. It

runs in this order:

1. **Clock and waveform.** `always #5 clk = ~clk` gives a 10 ns clock; all signals are dumped to `mult.vcd` from time 0.
2. **Reset.** `rst_n` = 0 for the first 2 rising edges (5 and 15 ns) and is released at the falling edge at 20 ns, so all 16 registers start at 0.
3. **All 256 inputs.** Two nested loops set `a` = 0..15 and `b` = 0..15. Each new pair is applied at a falling edge, half a cycle away from the rising edges, so it is never sampled while changing.
4. **Latency and check.** The testbench waits 2 rising edges (edge 1 loads `a_reg`/`b_reg`, edge 2 loads `p`), then 2 ns more, and compares `p` with `expected = a * b`. The `*` operator is used only here, as the reference. The comparison uses `!==`, so an unknown (`x`) output also counts as an error.
5. **Reporting.** Every mismatch prints `Mismatch: a=.. b=.. expected=.. got=..` and increments an error counter. After the last vector exactly one summary line is printed, `PASS: 256 vectors checked, 0 errors` or `FAIL: 256 vectors checked, N errors`.

Each vector takes 2 clock cycles: vector k is applied at $30 + 20k$ ns and checked at $47 + 20k$ ns, so the run ends at $47 + 20 \times 255 = 5147$ ns, matching `$finish called at 5147000 (1ps)`.

2.5 Simulation results

The design is simulated with:

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
```

To check that the testbench really catches errors (task 6), the sum in `full_adder.v` was temporarily changed from `a ^ b ^ cin` to `a ^ b` (carry-in ignored), simulated, and then restored with `git checkout rtl/full_adder.v`:

Run	RTL	Summary line
1	correct design	`PASS: 256 vectors checked, 0 errors`
2	bug injected: `sum = a ^ b`	`FAIL: 256 vectors checked, 95 errors`
3	restored (`git diff` empty)	`PASS: 256 vectors checked, 0 errors`

Table 4: RTL simulation results and the error-injection test.

In run 2 every wrong product is printed, e.g. `Mismatch: a=3 b=3 expected=9 got=5`. Only 95 of 256 fail because the bug matters only when a full adder has a carry-in of 1; products with no internal carry (e.g. $a = 0$, $b = 0$ or a power-of-two operand) are still correct. Full logs: `results/PartA/1_rtl_sim_pass.log` and `results/PartA/2_fail_test.log`.

2.6 Hand-worked example (task 5)

Roll number 2023102009 (last digit 9): $A = 9 + 5 = 14 = 1110_2$, $B = 7 = 0111_2$ ($A \neq 7$, so B stays 7). The expected product is $14 \times 7 = 98 = 0110\,0010_2$.

```
        1 1 1 0     A  = 14
      x 0 1 1 1     B  =  7
-----------------
        1 1 1 0     pp0 = A AND b0 (b0 = 1)              = 14
      1 1 1 0       pp1 = A AND b1 (b1 = 1), shifted 1   = 28
    1 1 1 0         pp2 = A AND b2 (b2 = 1), shifted 2   = 56
  0 0 0 0           pp3 = A AND b3 (b3 = 0), shifted 3   =  0
-----------------
  0 1 1 0 0 0 1 0   P = 14 + 28 + 56 + 0 = 98
```

The same product through the adder rows of Table 3 (bits written MSB first):

Row	pp_i	r_{i-1}	$\{c_i[4], s_i\}$	**Outputs**
0	1110	–	–	$p[0] = 0$, $r_0 = 0111$
1	1110	0111	1 0101 $(14 + 7 = 21)$	$p[1] = 1$, $r_1 = 1010$
2	1110	1010	1 1000 $(14 + 10 = 24)$	$p[2] = 0$, $r_2 = 1100$
3	0000	1100	0 1100 $(0 + 12 = 12)$	$p[3] = 0$, $p[7{:}4] = 0110$

Table 5: Row-by-row evaluation of 14×7 in the array.

$P = \{p[7{:}4], p[3], p[2], p[1], p[0]\} = \{0110, 0, 0, 1, 0\} = 0110\,0010_2 = 98$, the same as the shift-and-add result. The full working, including every full adder's inputs and outputs, is in `results/PartA/hand_example.txt`.

2.7 Waveform

In the exhaustive testbench the pair $a = 14$, $b = 7$ is vector $14 \times 16 + 7 = 231$, applied at $30 + 20 \times 231 = 4650$ ns. This shows the 2-cycle latency directly: the operands are captured at the first rising edge after they are applied (4655 ns), and the product 98 appears on `p` one rising edge later (4665 ns), so a result is available two clock cycles after its inputs.

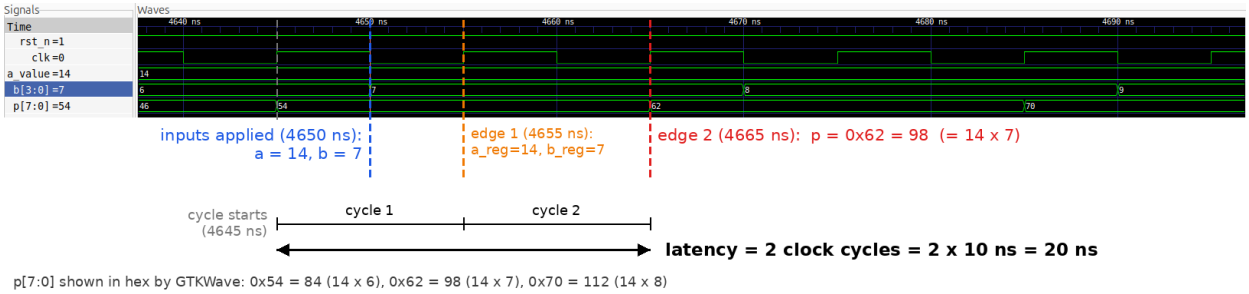


Figure 2: GTKWave view of `clk`, `rst_n`, the input values (`a_value` is the testbench variable that drives `a`), `b` and `p` for $A = 14$, $B = 7$. The inputs are applied at 4650 ns, captured into the input registers at rising edge 1 (4655 ns), and the product `0x62` $= 98$ appears on `p` at rising edge 2 (4665 ns): the latency is 2 clock cycles (20 ns). GTKWave shows `p` in hexadecimal (`0x54` $= 84$, `0x62` $= 98$, `0x70` $= 112$).

3 Part B: Logic Synthesis with Yosys

3.1 Synthesis script

Synthesis translates the behavioural Verilog into a *netlist*: a list of real SKY130 standard cells and the wires that connect them. Each cell is a fixed, pre-designed layout whose area and delay are given in the *Liberty* file (`.lib`). The key step is `abc`, which performs *technology mapping*: it chooses the combination of library gates that implements the logic with good area and delay. Because of `-flatten`, ABC sees the whole multiplier as one block instead of 12 separate full adders, so it can optimise across adder boundaries. Table 6 lists the steps of `synth/synth.tcl`.

Step	Command in `synth/synth.tcl`	Purpose
1	`yosys read_verilog rtl/full_adder.v rtl/mult_array.v`	Read the design
2	`yosys synth -top mult_array -flatten`	Generic synthesis; flatten the 12 full adders
3	`yosys dfflibmap -liberty $lib`	Map flip-flops to SKY130 cells
4	`yosys abc -liberty $lib`	Map the logic to SKY130 gates
5	`yosys opt_clean -purge`	Remove unused wires
6	`yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib`	Cell and area report
7	`yosys write_verilog -noattr synth/mult_array_netlist.v`	Gate-level netlist

Table 6: Steps of `synth/synth.tcl`, run with `yosys -c synth/synth.tcl`.

`$lib` is the typical-corner Liberty file `$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib`.

3.2 Part B results

All 16 flip-flops are `dfrtp_1`: a D flip-flop with an active-low asynchronous reset (the `_1` suffix is the drive strength). The remaining 57 cells implement the logic: 7 XOR/XNOR and 1 `maj3` cells for the adders' sums and carries, 6 AND and 19 NAND/NOR gates, 20 compound AND-OR / OR-AND gates (such as `a21oi` = $\overline{A_1A_2 + B_1}$), 3 isolation-type cells and 1 inverter (Table 8).

Metric	Value
Total number of cells	73
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`: D flip-flop with active-low asynchronous reset)
Total cell area (μm^2)	805.77
Flip-flop area (μm^2)	400.38 = 16 × 25.024
Flip-flop area (% of total)	49.69 %
The 3 most-used cell types and their counts	`dfrtp_1` × 16, `nand2_1` × 11, `a21oi_1` × 4
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 7: Part B results table (from `synth/mult_array_stat.rpt`).

3.3 Full cell breakdown

Group	Cells (`sky130_fd_sc_hd__...`)	Count	Area (μm^2)
Flip-flops	`dfrtp_1` ×16	16	400.384
Sum logic (XOR/XNOR)	`xnor2_1` ×4, `xnor3_1` ×2, `xor3_1` ×1	7	103.850
Carry logic (majority)	`maj3_1` ×1	1	10.010
AND gates	`and2_0` ×4, `and3_1` ×1, `and4_1` ×1	6	40.038
NAND / NOR	`nand2_1` ×11, `nand3_1`, `nand4_1` ×4, `nor2_1`, `nor2b_1`, `nor3_1`	19	86.334
AND-OR / OR-AND compound	`a21oi_1` ×4, `a21boi_0` ×2, `o21ai_0` ×2, `o31a_1` ×2, `a21o_1`, `a22oi_1`, `a32oi_1`, `o2111a_1`, `o2111ai_1`, `o21a_1`, `o22ai_1`, `o311ai_0`, `o31ai_1`, `or3_1`	20	148.651
Isolation-type cells	`lpflow_inputiso1p_1`, `lpflow_isobufsrc_1` ×2	3	18.768
Inverters	`clkinv_1`	1	3.754
Total (31 cell types)		**73**	**805.773**

Table 8: All cells in the synthesised netlist, grouped by function (cells without a count appear once).

3.4 Gate-level (netlist) simulation

Synthesis could, in principle, break the design (for example by mapping a register wrongly). To check this, we ran the *same* testbench on the synthesised netlist instead of the RTL. The simulator is given the netlist together with the SKY130 cell models (`primitives.v` and `sky130_fd_sc_hd.v`): `-DFUNCTIONAL` uses the logic-only version of each cell, and `-DUNIT_DELAY=#1` adds a tiny delay to every cell so that signals change in a clean order.

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v \
    synth/mult_array_netlist.v
vvp gls
```

Result: `PASS: 256 vectors checked, 0 errors`. The netlist gives the same product as the RTL for every input, so synthesis did not change the behaviour.

4 Part C: Static Timing Analysis with OpenSTA

4.1 STA script and constraints

Static timing analysis (STA) finds the maximum speed without running any simulation: it traces every path from one flip-flop to the next and adds up the cell delays taken from the Liberty file. Data launched at one rising edge must arrive and settle before the next rising edge, which gives the setup condition

$$t_{clk \to Q} + t_{logic} + t_{setup} \leq T,$$

where $t_{clk \to Q}$ is the delay of the launching flip-flop, t_{logic} the delay through the gates and t_{setup} the time the data must be stable before the capturing edge. The *slack* of a path is its required time ($T - t_{setup}$) minus its arrival time: positive slack means the path meets timing, negative means it fails. The smallest slack in the design is the worst negative slack (WNS), and the path that has it is the *critical path*. `sta/sta.tcl` runs the steps below.

Step	Command in `sta/sta.tcl`	Purpose
1	`read_liberty $lib` (tt_025C_1v80)	Read the cell timing library
2	`read_verilog synth/mult_array_netlist.v; link_design mult_array`	Read and link the netlist
3	`create_clock -name clk -period $clock_period [get_ports clk]`	Clock with period T (10 ns, or `STA_PERIOD`)
4	`set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [get_ports clk]]`	Input arrival time
5	`set_output_delay 1.0 -clock clk [all_outputs]`	Output required time
6	`report_worst_slack -max`	Worst setup slack
7	`report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt`	Critical path report

Table 9: Steps of `sta/sta.tcl`, run with `sta -exit sta/sta.tcl`.

The analysis is pre-layout: the clock is ideal (no skew or insertion delay) and there is no wire delay.

4.2 Finding the maximum frequency

```
$ sta -exit sta/sta.tcl                    (T = 10.00 ns)
worst slack max 7.40                       (report: +7.404 ns)

T_min = T - WNS   = 10.000 - 7.404 = 2.596 ns
F_max = 1000/T_min = 1000 / 2.596   = 385.21 MHz

$ STA_PERIOD=2.60 sta -exit sta/sta.tcl    (T = T_min rounded to 0.01 ns)
worst slack max 0.00                       (report: +0.004 ns)
```

At $T = 2.60$ ns the slack is +0.004 ns, effectively zero, which confirms T_{min}. The small positive value comes from rounding 2.596 ns up to 2.60 ns.

4.3 Part C results

Metric	Value
Worst slack at $T = 10$ ns (ns)	+7.404
T_{min} (ns)	2.596
F_{max} (MHz)	385.21
Slack at $T = T_{min}$ (ns)	+0.004 at $T = 2.60$ ns (≈ 0)
Critical path start point	`_118_`: input register `b_reg[0]` (`dfrtp_1`, CLK→Q)
Critical path end point	`_112_`: output register `p[6]` (`dfrtp_1`, D = `fa33.sum`)
Number of cells on the critical path	10: 2 flip-flops + 8 combinational cells
Latency (ns) = $2 \times T_{min}$	5.192
Throughput (million results per second)	385.21 (one result per clock cycle)

Table 10: Part C results table (typical corner, pre-layout).

4.4 Critical path

#	Instance	Cell	Output equals	Role in the multiplier	Incr (ns)	Arrival (ns)
1	`_118_/Q`	`dfrtp_1`	`b_reg[0]`	launch flip-flop (input register)	0.395	0.395
2	`_083_/X`	`and2_0`	$pp_0[0]$	AND gate $a_0 b_0$, reused in the row-1 carry logic	0.212	0.607
3	`_055_/X`	`or3_1`	$\overline{c_1[2]} + pp_0[0]$	part of the carry of FA[1,1]	0.361	0.968
4	`_059_/X`	`and4_1`	$pp_2[1] \cdot c_2[1]$	carry term of FA[2,1]	0.215	1.183
5	`_070_/X`	`o2111a_1`	$pp_2[2] \cdot c_2[2]$	carry term of FA[2,2]	0.234	1.417
6	`_076_/Y`	`o21ai_0`	$\overline{c_2[3]}$	carry out of FA[2,2] (inverted)	0.216	1.634
7	`_090_/X`	`xnor3_1`	$\overline{s_2[3]}$	sum of FA[2,3] (inverted)	0.274	1.907
8	`_091_/X`	`maj3_1`	$\overline{c_3[3]}$	carry out of FA[3,2] (inverted)	0.375	2.282
9	`_092_/X`	`xor3_1`	$s_3[3]$	sum of FA[3,3] → `p[6]`	0.178	2.460
10	`_112_/D`	`dfrtp_1`	`p[6]`	capture flip-flop, setup 0.135 ns	0.000	2.460

Table 11: Critical path from `sta/mult_array_timing.rpt` ($T = 2.60$ ns). Cell roles were found by simulating the netlist for all 256 values of (`a_reg`, `b_reg`) and matching each net's truth table with the RTL signals $pp_i[j]$, $s_i[j]$, $c_i[j]$ (a bar means the inverted signal).

Data arrival = 2.460 ns; data required = 2.600 − 0.135 = 2.465 ns; slack = +0.004 ns. At $T = 10$ ns the same path has slack 9.865 − 2.460 = +7.404 ns.

Other paths. All other paths are short. An input-to-register path is only the 1.0 ns input delay straight into a flip-flop, and a register-to-output path is only CLK→Q plus the 1.0 ns output delay. The report also checks the *recovery* time of the asynchronous reset (`rst_n` must be released a minimum time before a clock edge): its slack is +9.228 ns at 10 ns and +1.828 ns at 2.60 ns. Neither limits the clock, so the register-to-register path through the adder array sets T_{min}.

4.5 Comparison with the $2N = 8$ full-adder estimate

Full-adder stages. The path passes through 6 full adders: FA[1,1] → FA[2,1] → FA[2,2] → FA[2,3] → FA[3,2] → FA[3,3]. It follows the carry along row 1, down into row 2, along row 2, and then through the sums and carries of rows 2 and 3 into product bit 6. Seven of the 8 logic cells belong to these adders; only `_083_` (`and2_0`) is a plain AND gate.

Delay per stage. The 8 logic cells add 2.065 ns (2.460 − 0.395), about 0.26 ns per cell. Each full adder costs only about one cell because the path uses either its carry or its sum, never both: a carry is one `maj3` or AND-OR cell, and a sum is one `xor3`/`xnor3` cell.

Comparison with 2N. Six full-adder stages in 8 cells agree well with the estimate of about $2N = 8$ full-adder delays, which is only an approximation of the longest chain. The match cannot be exact because `synth -flatten` removes the `full_adder` boundaries and ABC merges AND gates and parts of several adders into single compound cells.

5 Part D: Physical Design with LibreLane

5.1 The flow

LibreLane 3.1.0.dev3 (Classic flow) takes the RTL all the way to a layout that could be sent for manufacture, running about 76 automatic steps. It was run inside the IIC-OSIC-TOOLS container with the SKY130 PDK selected (`export PDK=sky130A STD_CELL_LIBRARY=sky130_fd_sc_hd`, then `librelane config.json`), because the container's default PDK is IHP. Table 12 summarises the main stages.

Stage	What it does
Synthesis	Converts the RTL into SKY130 standard cells, as in Part B (LibreLane uses its own Yosys script).
Floorplan	Fixes the die at 150 μm × 150 μm and the core (the area for cells) at 17759.5 μm^2, and creates 47 rows of cell sites.
Power grid	Builds the VPWR (1.8 V) and VGND (0 V) supply rails and straps.
Tap insertion	Places well-tap cells at regular spacing in every row.
Placement	Puts every cell on a legal site in the rows; buffers are added on the I/O ports and on high-fanout nets.
CTS	Clock-tree synthesis: buffers the clock so it reaches all 16 flip-flops at nearly the same time (the difference is the *skew*).
Timing repair	Resizes gates and adds buffers where the timing estimate shows a violation.
Routing	Draws the metal wires and vias between all pins.
Extraction	Calculates the real resistance and capacitance of every wire.
Signoff	STA with the real wires at 9 corners, DRC (Magic and KLayout), LVS (Netgen), antenna and IR-drop checks.
GDSII	Writes the final layout file that would be sent to the foundry.

Table 12: Main stages of the LibreLane Classic flow.

`pnr/config.json` contains exactly the six required keys:

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v"],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 7.0,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

The core box is (5.52, 10.88) to (144.44, 138.72) μm inside the fixed 150 μm × 150 μm die.

5.2 Timing closure: every attempt

The first clock period is $1.2 \times T_{min} = 1.2 \times 2.596 = 3.115$ ns, rounded up to the next 0.5 ns, i.e. 3.5 ns. The 20 % margin over the Part C result is meant to cover the delay of real wires and the clock tree. After every run that failed setup timing, the period was increased by 0.5 ns.

Corners. Manufacturing and operating conditions vary, so signoff STA checks 9 *corners*: three process/voltage/temperature conditions — typical (tt, 1.80 V, 25 °C), slow (ss, 1.60 V, 100 °C) and fast (ff, 1.95 V, −40 °C) — each with minimum, nominal and maximum wire RC. The chip must meet timing at all of them. Setup is hardest at the slow corner (slowest cells) and hold at the fast corner.

Run	Period (ns)	Slow-corner slack estimated before routing (ns)	Cells added by timing repair	Final setup slack (ns)	Hold slack (ns)	Wire-length (μm)	Result
1	3.5	−2.565	4	−2.154	+0.100	2400	FAIL
2	4.0	−2.065	4	−1.654	+0.147	2400	FAIL
3	4.5	−1.565	4	−1.154	+0.147	2400	FAIL
4	5.0	−1.065	4	−0.654	+0.147	2402	FAIL
5	5.5	−0.565	4	−0.155	+0.147	2393	FAIL
6	6.0	−0.065	4	−0.107	+0.148	2376	FAIL
7	6.5	+0.435	0	−0.136	+0.141	2362	FAIL
8	**7.0**	+0.935	0	**+0.364**	+0.141	2362	**PASS**

Table 13: All LibreLane runs. Final setup slack is `setup__ws`, the worst over the 9 corners (always the slow corner). Every run has 0 DRC and 0 LVS errors, and the typical corner passes in every run (+0.587 ns at 3.5 ns). Per-run configs and metrics are in `results/PartD/attempts/`.

Why the slack changes unevenly. During placement LibreLane checks timing with *estimated* wire delays, and the timing-repair step only acts if that estimate shows a violation. The real wire delays are known only after routing. The synthesised logic (67 cells) is identical in every run, so the runs differ only in what the repair step does:

- **3.5–6.0 ns:** the slow-corner estimate is negative, so the repair step adds the same 4 cells every time (3 `buf_4` re-buffers and 1 `dlygate4sd3_1` hold-delay cell). The layout is otherwise the same, so up to 5.5 ns the final slack improves by exactly 0.5 ns per step; at 6.0 ns it ends at −0.107 ns.
- **6.5 ns:** the estimate is already positive (+0.435 ns), so the repair step adds nothing. The real routed wires turn out about 0.57 ns slower than estimated, so the final slack is −0.136 ns, slightly worse than at 6.0 ns.
- **7.0 ns:** the estimate (+0.935 ns) leaves enough room for the real wires, and the run passes with +0.364 ns.

5.3 Part D results

Metric	Value
CLOCK_PERIOD used (ns)	7.0
Number of runs needed	8 (3.5 → 7.0 ns in 0.5 ns steps)
Core area (μm^2)	17759.5 (die area 22500 = 150 × 150 μm)
Cells after synthesis (from Part B)	73 (16 flip-flops + 57 logic cells)
Cells after place and route	1895 instances (334 excluding decap/fill)
Kinds of cells added	buffers ×22: `buf_2` ×17 (I/O ports), `clkdlybuf4s50_1` ×2 (reset fanout), `clkbuf_16` ×3 (clock tree); tap ×245 (`tapvpwrvgnd_1`); decap ×1314 (`decap_*`); fill ×247 (`fill_*`)
Wirelength (μm)	2362 (552 vias)
Setup slack (ns)	+0.364 (worst corner `max_ss_100C_1v60`)
Post-route F_{max} (MHz)	150.70 = 1000 / (7.0 − 0.364)
F_{max} from Part C (MHz)	385.21
DRC errors	0 (Magic 0, KLayout 0, router 0)
LVS errors	0 (Netgen: circuits match uniquely)

Table 14: Part D results table (final run, CLOCK_PERIOD = 7.0 ns, from `pnr/metrics.json`).

5.4 Timing at every signoff corner

Corner (process, V, T)	Wire RC	Setup slack (ns)	Hold slack (ns)	Implied F_{max} (MHz)
tt, 1.80 V, 25 °C	min	3.729	0.363	305.7
	nom	3.699	0.370	302.9
	max	3.678	0.374	301.1
ss, 1.60 V, 100 °C	min	0.444	0.822	152.5
	nom	0.406	0.815	151.7
	max	**0.364**	0.808	**150.7**
ff, 1.95 V, −40 °C	min	4.735	0.141	441.6
	nom	4.726	0.145	439.8
	max	4.719	0.148	438.5

Table 15: Setup and hold slack at each of the 9 signoff corners (final run). Implied F_{max} = 1000 / (7.0 − setup slack).

The worst setup path is register-to-register (`timing__setup_r2r__ws` = 0.364 ns), from `_102_` (`a_reg[0]`) to `_100_` (D = `fa33.sum`, output `p[6]`): the same output bit as the Part C path. Other signoff results of the final run:

- The typical corner has 3.699 ns of setup slack to spare; only the slow corner is tight.
- Hold slack is positive everywhere; it is smallest (+0.141 ns) at the fast corner, as expected.
- Clock skew is 0.26–0.28 ns at every corner.
- 0 antenna violations, 0 max-slew, max-capacitance and max-fanout violations, 0 disconnected pins.
- The worst IR drop (voltage lost along the power rails) is 0.77 mV.

- Total power at the typical corner is 0.287 mW.

5.5 Post-route F_{max} compared with Part C

Post-route $F_{max} = 1000/(7.0 - 0.364) = 150.70$ MHz, about 2.6 times lower than the 385.21 MHz of Part C. Two effects combine:

1. **Corner** (about 2×). Part C used only the typical corner, while `setup__ws` is set by the slow corner (1.60 V, 100 °C), where the cells are about twice as slow. The same layout at the typical corner gives $1000/(7.0 - 3.699) = 302.9$ MHz.
2. **Physical effects** (about 27 % at the same corner). The minimum period at the typical corner grows from 2.596 ns to 3.301 ns (Table 16).

Component	Part C (ns)	Post-route, tt (ns)	Added (ns)
Launch flip-flop CLK→Q	0.395	0.472	+0.077 (real wire load)
Logic cells + wires	2.065 (8 cells)	2.439 (9 cells)	+0.374 (wire RC, LibreLane's netlist)
Clock uncertainty	0	0.250	+0.250 (LibreLane default)
Clock skew (0.336 − 0.302 − CRPR 0.018)	0	0.016	+0.016
Library setup time	0.135	0.124	−0.011
Minimum period	**2.596**	**3.301**	**+0.705**

Table 16: Where the extra delay comes from at the typical corner (post-route report of path `a_reg[0]` → `p[6]`).

5.6 Cells added by place and route

Part B synthesis gave **73 cells**; after place and route there are **1895 cells** (`design__instance__count`), about 26 times more. The added cells were found by name in the LibreLane cell-frequency report:

Name contains	Cells	Count	Why the flow adds them
buf	buf_2 ×17 (input1–9, output10–17)	17	I/O port buffers: one on each of the 9 inputs (rst_n, a, b) and 8 outputs (p); they isolate the core logic from unknown external drivers and loads.
	clkdlybuf4s50_1 ×2 (fanout18, fanout19)	2	Fanout buffers: split the reset net, which drives all 16 flip-flops, into two branches (10 + 6) to meet slew and load limits.
	clkbuf_16 ×3	3	Clock-tree synthesis: deliver the clock to the 16 flip-flops with low skew.
tap	tapvpwrvgnd_1	245	Tie the n-well to VPWR and the substrate to VGND at regular intervals to prevent latch-up.
decap	decap_3 ×130, decap_4 ×49, decap_6 ×175, decap_8 ×55, decap_12 ×905	1314	Decoupling capacitors between VPWR and VGND that reduce supply noise and IR drop.
fill	fill_1 ×215, fill_2 ×32	247	Close the gaps so every row is continuous (wells, implants, power rails) and density rules are met.
Added in total		**1828**	
Logic (51) + flip-flops (16), from LibreLane's own synthesis		67	
Total after place and route		**1895**	

Table 17: Instance classes in the final layout.

The logic part is 67 cells rather than Part B's 73 because LibreLane runs its own Yosys synthesis with a different script; $67 + 1828 = 1895$. Without decap and fill cells there are 334 standard-cell instances with an area of 1224.92 μm^2, a utilisation of only 6.9 % of the 17759.5 μm^2 core. The fixed 150 μm × 150 μm die is much larger than this design needs, which is why tap, decap and fill cells make up almost all of the instances.

5.7 Final layout

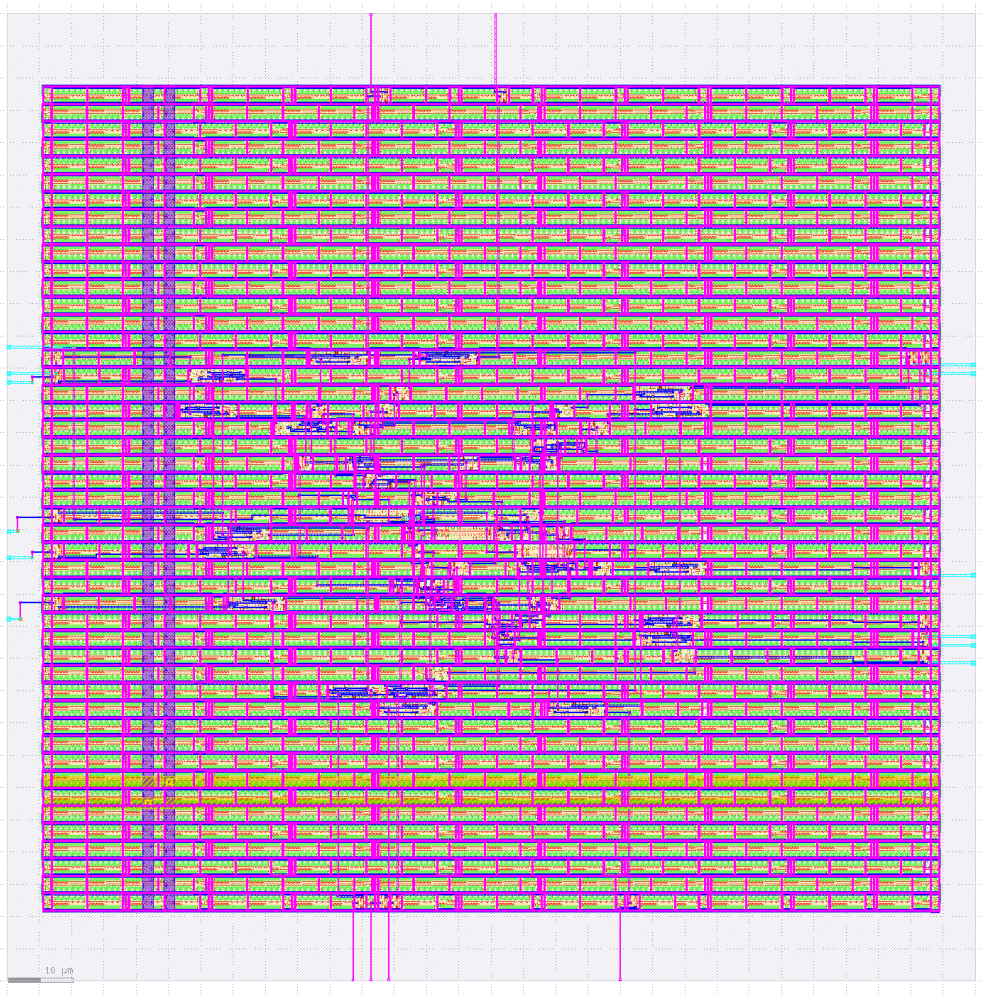


Figure 3: Whole die (150 μm $\times$ 150 μm) in KLayout: I/O pins on the die edges, 47 standard-cell rows with the power grid, and the routed logic in the centre of the core.

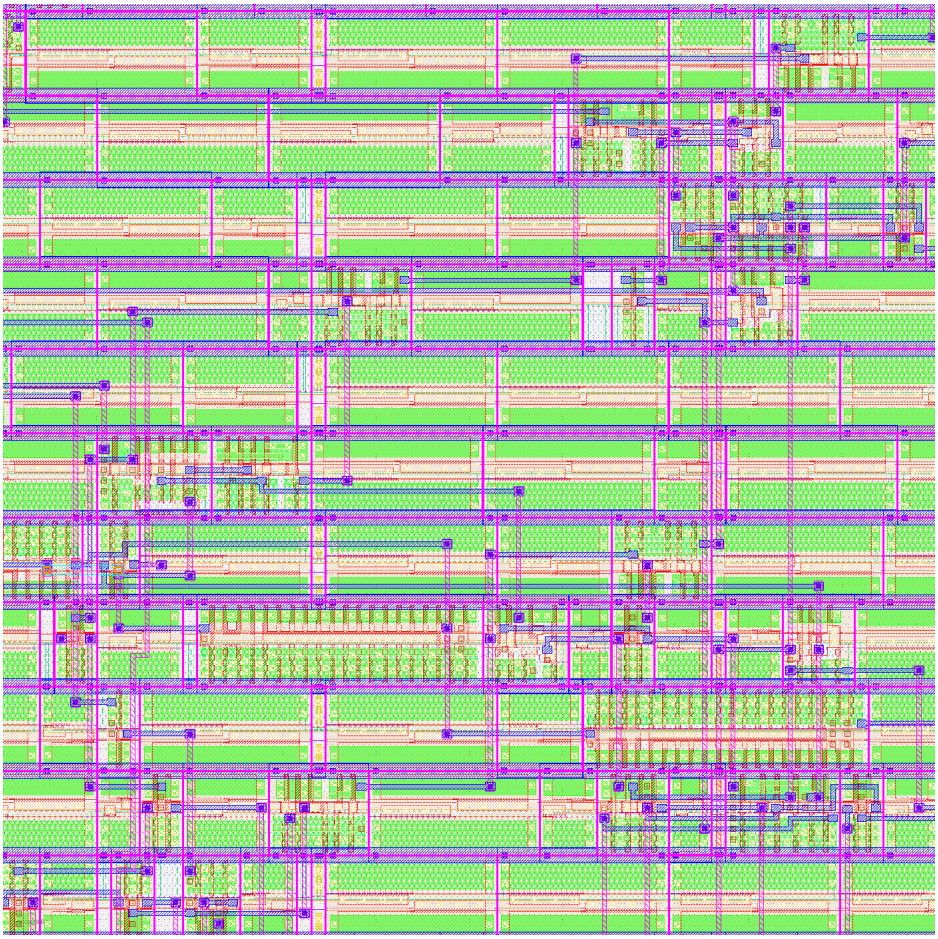


Figure 4: Zoomed view (30 μm $\times$ 30 μm): standard-cell rows bounded by the horizontal power rails, with routed metal-1/metal-2 wires and vias connecting the cell pins.

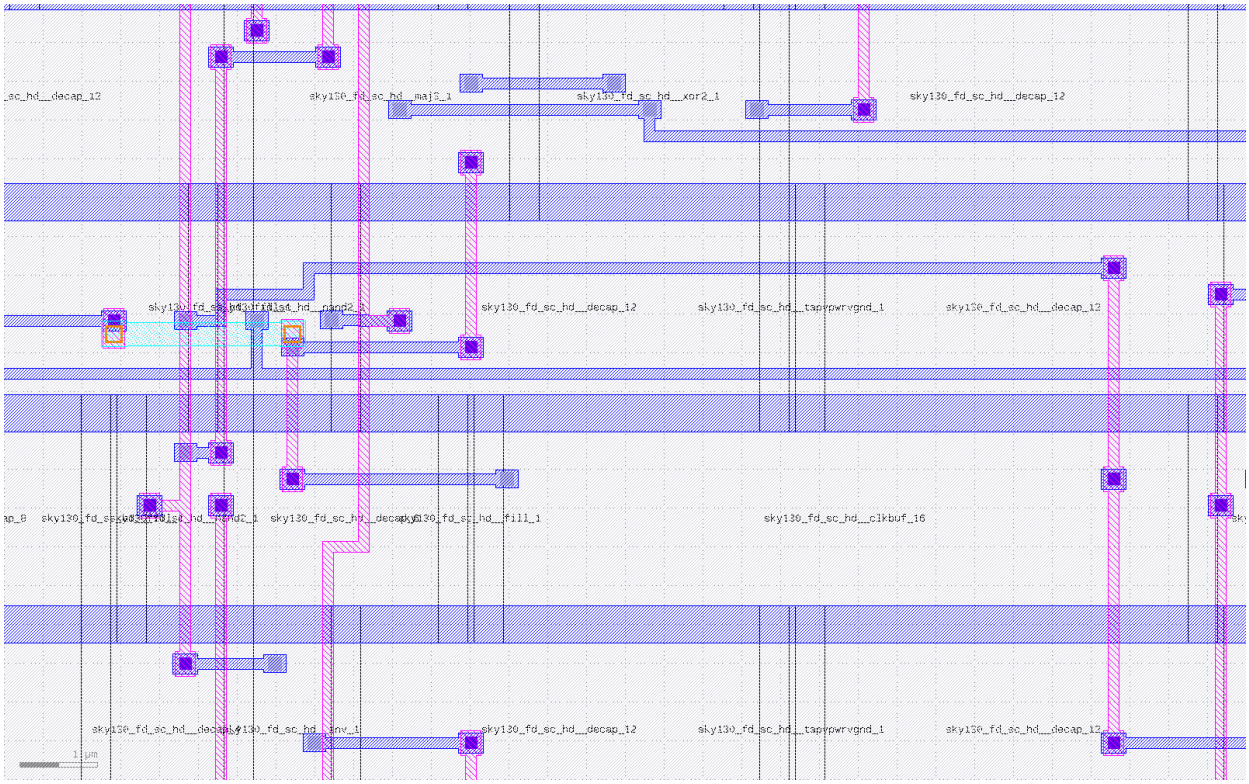


Figure 5: Deeper zoom (16 μm $\times$ 10 μm) with every standard cell drawn as a labelled box: logic cells (`maj3_1`, `xor2_1`, `nand2_1`, `inv_1`) next to cells added by the flow (`decap_12`, `tapvpwrvgnd_1`, `fill_1`, `clkbuf_16`), the power rails between rows and the routed wires.

6 Answers to the Report Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic? Explain why the number of cells is not simply 16 + 12.

Synthesis produced 73 cells. 16 of them are flip-flops, so the logic uses **57 cells** (405.39 μm^2), not 28. "AND gate" and "full adder" are blocks in our RTL, not single library cells. Because of `-flatten`, Yosys removes the `full_adder` boundaries and ABC rebuilds the whole logic from whichever SKY130 cells give a good area and delay (Table 8). The count changes for four reasons:

- **One full adder needs at least two cells.** Its sum becomes an XOR/XNOR cell (7 in total) and its carry a `maj3` cell or a small NAND/AND-OR network. ABC cannot use SKY130's two-output full-adder cell `fa_1` (the log reports "Detected 9 multi-output cells"), so 12 full adders alone need about 24 cells or more.
- **AND gates are merged into other cells.** Most partial-product ANDs disappear inside compound gates such as `a21oi`, `a21boi`, `o2111a` and `nand4`; only 6 plain AND cells remain.
- **Constant inputs make some adders simpler.** The first carry of every row and the top bit of r_0 are always 0, so those full adders shrink to half adders.
- **A few inverting cells are needed** (`clkinv`, `nor2b`) to match signal polarities.

The exact number also depends on how the RTL is written and on the tool version, because ABC is a heuristic. For example, writing the carry as `(a&b)|(a&cin)|(b&cin)` instead of our `(a&b)|(b&cin)|(a&cin)` (the same logic) gives 61 cells with the same tools. In every case the count is well above 28.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?

The critical path starts at the input register `b_reg[0]` (`_118_`) and ends at the output register `p[6]` (`_112_`), whose input is the sum of the last full adder, FA[3,3]. It has **10 cells**: the launching flip-flop, 8 logic cells (`and2` → `or3` → `and4` → `o2111a` → `o21ai` → `xnor3` → `maj3` → `xor3`) and the capturing flip-flop. The data arrives after 2.460 ns, which gives $T_{min} = 2.596$ ns.

By matching every net with the RTL signals (Table 11), the path passes through **6 full adders**: FA[1,1] → FA[2,1] → FA[2,2] → FA[2,3] → FA[3,2] → FA[3,3]. It follows the carry along row 1 and row 2 and then the sums and carries of rows 2 and 3 into $p[6]$. Seven of the 8 logic cells belong to these adders, at about 0.26 ns per cell. This agrees well with the estimate of about $2N = 8$ full-adder delays; each adder costs only one cell because the path uses either its carry or its sum, never both.

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

Flip-flops take 400.38 of 805.77 μm^2, which is **49.69 %** of the cell area, even though they are only 16 of the 73 cells (22 %). There are two reasons:

- **A flip-flop is a big cell.** One `dfrtp_1` is 25.02 μm^2: 6.7 times a `nand2_1` (3.75 μm^2), about 3.5 times the average logic cell (7.1 μm^2), and larger than any logic cell in the design. A flip-flop with asynchronous reset contains two latches plus clock and reset circuitry, a few dozen transistors compared with 4 for a NAND2.
- **Scaling.** The array logic grows with N^2 (16 ANDs and 12 adders), but the registers grow

only with $4N$ (16 flip-flops). At $N = 4$ the logic is still small, so the registers dominate. At 8 bits the flip-flop share drops to about 32 % (Q6).

Q4. Why is the post-route F_{max} lower than the F_{max} from Part C? Which kinds of cells did the flow add, and why is each kind needed?

The post-route F_{max} is **150.70 MHz**, against 385.21 MHz from Part C, about 2.6 times lower. The two numbers are measured under very different conditions:

1. **Corner.** Part C used only the typical library (tt, 1.80 V, 25 °C). LibreLane signs off at 9 corners and reports the worst one, the slow corner (ss, 1.60 V, 100 °C), where cells are about twice as slow. The same layout reaches 302.9 MHz at the typical corner, so the corner alone explains a factor of about 2.
2. **Real wires.** Part C had no wire delay. After routing, 2362 μm of wire and 552 vias add resistance and capacitance: on the critical path the logic delay grows by 0.374 ns and CLK→Q by 0.077 ns.
3. **Real clock.** Part C used an ideal clock. The real clock tree has insertion delay and 0.26–0.28 ns of skew between flip-flops.
4. **Stricter constraints.** LibreLane adds 0.25 ns of clock uncertainty and a 5 % timing derate to every path. (Its input and output delays are also larger, 20 % of the period = 1.4 ns, but these only affect the I/O paths, not our register-to-register critical path.)
5. **Different netlist.** LibreLane re-synthesises the RTL with its own script (67 cells), so the post-route critical path, `a_reg[0]` → `p[6]` with 9 logic cells, is not the same as in Part C.

At the same typical corner, items 2–5 add 0.705 ns to the minimum period (Table 16).

The flow added these cells (Table 17):

- **Buffers (22):** 17 `buf_2` I/O port buffers, one on each input and output pin, which separate the core from outside drivers and loads; 2 `clkdlybuf4s50_1` fanout buffers, which split the reset net that drives all 16 flip-flops; and 3 `clkbuf_16` clock-tree buffers, which deliver the clock to the 16 flip-flops with low skew.
- **Tap cells (245):** connect the n-well to VPWR and the substrate to VGND at regular spacing to prevent latch-up.
- **Decap cells (1314):** small capacitors between VPWR and VGND that reduce supply noise.
- **Fill cells (247):** fill the gaps so that wells, implants and power rails stay continuous along every row, which DRC requires.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC (design rule check) checks the *shapes* in the layout against the foundry's rules for every layer: minimum width and spacing, enclosure and overlap, via rules, metal density and antenna ratios. For example, two metal wires placed closer than the minimum spacing might short together when printed. The rules describe what the factory can make reliably, so a violation can cause shorts, open wires or very low yield.

LVS (layout versus schematic) checks the *circuit*. It extracts the transistors and their connections from the layout and compares them with the netlist: the same devices, the same nets and the same pins. For example, a missing via or two nets accidentally touching would make the layout a different circuit from the one we designed.

Each check catches what the other cannot: a layout can follow every drawing rule but be the wrong circuit, or be the right circuit but impossible to manufacture. Masks are very expensive and a chip cannot be fixed after it is made, so both must pass first. Our final run has **0 DRC errors** in both

Magic and KLayout (the router's intermediate violations, $28 \to 3 \to 3 \to 0$, were all fixed), **0 LVS errors** in Netgen ("Circuits match uniquely") and 0 antenna violations.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

Quantity	Formula	N = 4 (ours)	N = 8 (estimate)	Change
AND gates	N^2	16	64	×4
Full adders	$N(N-1)$	12	56	×4.67
Flip-flops	$4N$	16	32	×2
Critical path	$\approx 2N$ full-adder delays	8	16	×2

Table 18: Scaling from 4 to 8 bits.

Working. From Part C, the logic on the critical path takes 2.065 ns, and the flip-flop overhead is 0.395 ns (CLK→Q) + 0.135 ns (setup) = 0.530 ns. The logic delay grows with the path length ($2N$), so it doubles from 4 to 8 bits, while the flip-flop overhead stays the same:

$$T_{min}(\text{8-bit}) \approx 0.530 + 2 \times 2.065 = 4.66 \text{ ns}, \qquad F_{max}(\text{8-bit}) \approx 1000/4.66 \approx 215 \text{ MHz}.$$

As a cross-check with our measured path: for $N = 4$ it crosses 6 full adders ($2N - 2$), about 0.344 ns each. The same pattern gives 14 adders for $N = 8$, so $T_{min} \approx 0.530 + 14 \times 0.344 = 5.35$ ns (187 MHz). The 8-bit multiplier would therefore run at roughly **185–215 MHz** instead of 385 MHz (typical corner, before layout).

So the number of full adders grows about 4.7 times (with N^2), but the critical path grows only about 2 times (with N), and T_{min} rises less than 2 times (2.60 → 4.7–5.4 ns) because the flip-flop overhead does not change. For area, scaling the logic by the number of blocks, $(64 + 56)/(16 + 12) \approx 4.3$, gives about 1740 μm^2 of logic; with 32 flip-flops (801 μm^2) the total is about 2540 μm^2, and the flip-flop share falls to about 32 %.

7 Individual Contributions

The work was split equally between the two team members. Following the marking scheme of the assignment, each member was responsible for 50 of the 100 marks: Parts A and C (30 + 20 marks) for one member, and Parts B and D (20 + 30 marks) for the other.

Ande Karthik (2023102009) designed and verified the RTL in Part A: the full adder, the array multiplier, the exhaustive self-checking testbench, the error-injection test, the hand-worked example and the waveform. He also performed the static timing analysis in Part C: he wrote `sta.tcl`, found T_{min} and F_{max}, and traced the critical path through the full-adder stages. He answered Q2 and Q6, kept the command log in the README up to date, and wrote and typeset this report.

Kalluri Roshan Lal (2023102010) performed the logic synthesis in Part B: he wrote `synth.tcl`, ran the gate-level simulation and analysed the cell usage and area. He also completed the physical design in Part D: `config.json`, the 8 LibreLane runs and the analysis of why their timing changed, the cells added by the flow, the DRC/LVS signoff and the layout screenshots. He drew the block diagram, answered Q1, Q3, Q4 and Q5, and reviewed the report and cross-checked its results.